

Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1 Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

<https://archive.ics.uci.edu/dataset/267/banknote+authentication>

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
 - Skewness
 - Curtosis
 - Entropy
- Target Class
- 0 – Authentic Banknote
 - 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output

Dataset summary and statistical information.

Task 2: Exploratory Data Analysis

Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation

Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training

Display

- Epoch Number
- Misclassified Samples
- Updated Weights

- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7 Source Code

The complete source code developed for this experiment is publicly available on GitHub. The repository contains the implementation of the Single Layer Perceptron from scratch, including dataset preprocessing, exploratory data analysis, model training, evaluation, visualization, and additional logic gate experiments.

The repository includes the following components:

- Single Layer Perceptron implementation from scratch using Python
- Banknote Authentication dataset
- Exploratory Data Analysis (EDA)
- Training and evaluation of the perceptron model
- Decision boundary visualization
- Training error, weight, and bias evolution plots
- Learning rate comparison
- Confusion matrix
- Logic gate implementations (AND, OR, NOT, and XOR)
- Complete Jupyter Notebook with generated results

The complete source code can be accessed at:

<https://github.com/PandiKabilesh2006/CS3807-Deep-Learning-Laboratory>

8. Mandatory Plots

8.1 Feature Histograms

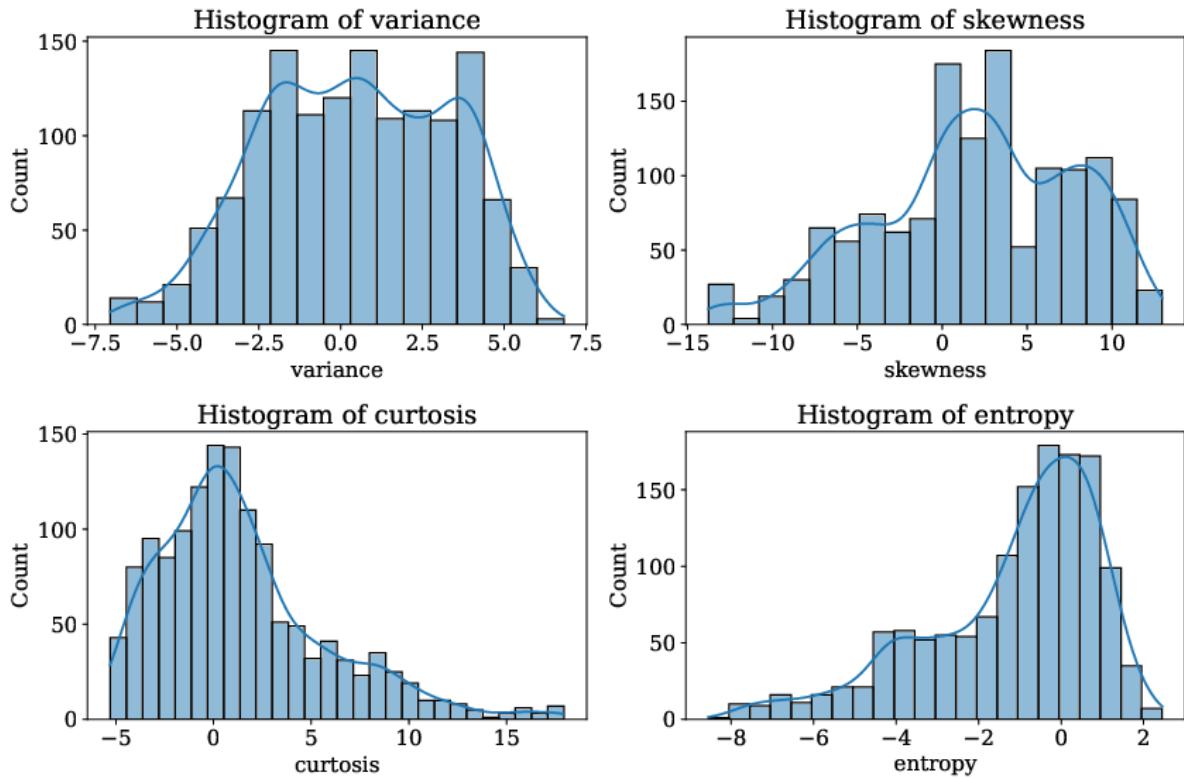


Figure 1: Histograms of the Banknote Authentication features.

Inference: The histogram plots represent the frequency distribution of the numeric features in the data set. There are features whose distributions demonstrate skewness, which means that the distribution of the features does not have symmetry. No presence of extreme outliers implies that the data set can be used to train the perceptron model.

8.2 Correlation Heatmap

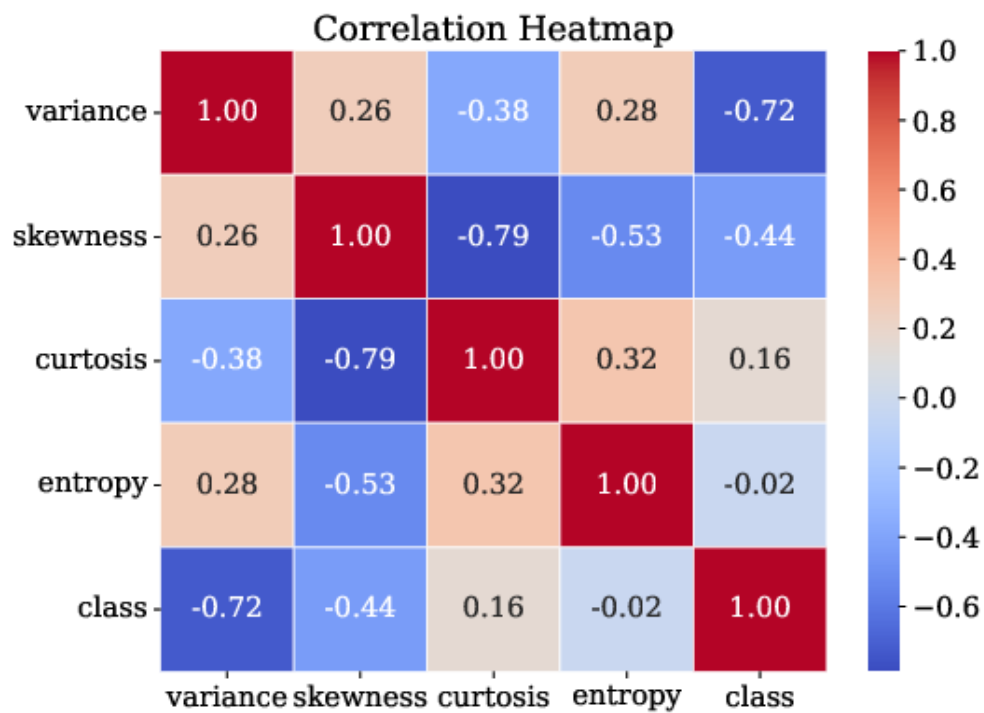


Figure 2: Correlation heatmap of the input features.

Inference: This graph demonstrates the correlation between two variables. A strong positive or negative correlation implies that some variables have a close relationship among themselves, while a weak correlation implies that the variables provide independent information to the classifier.

8.3 Scatter Plot

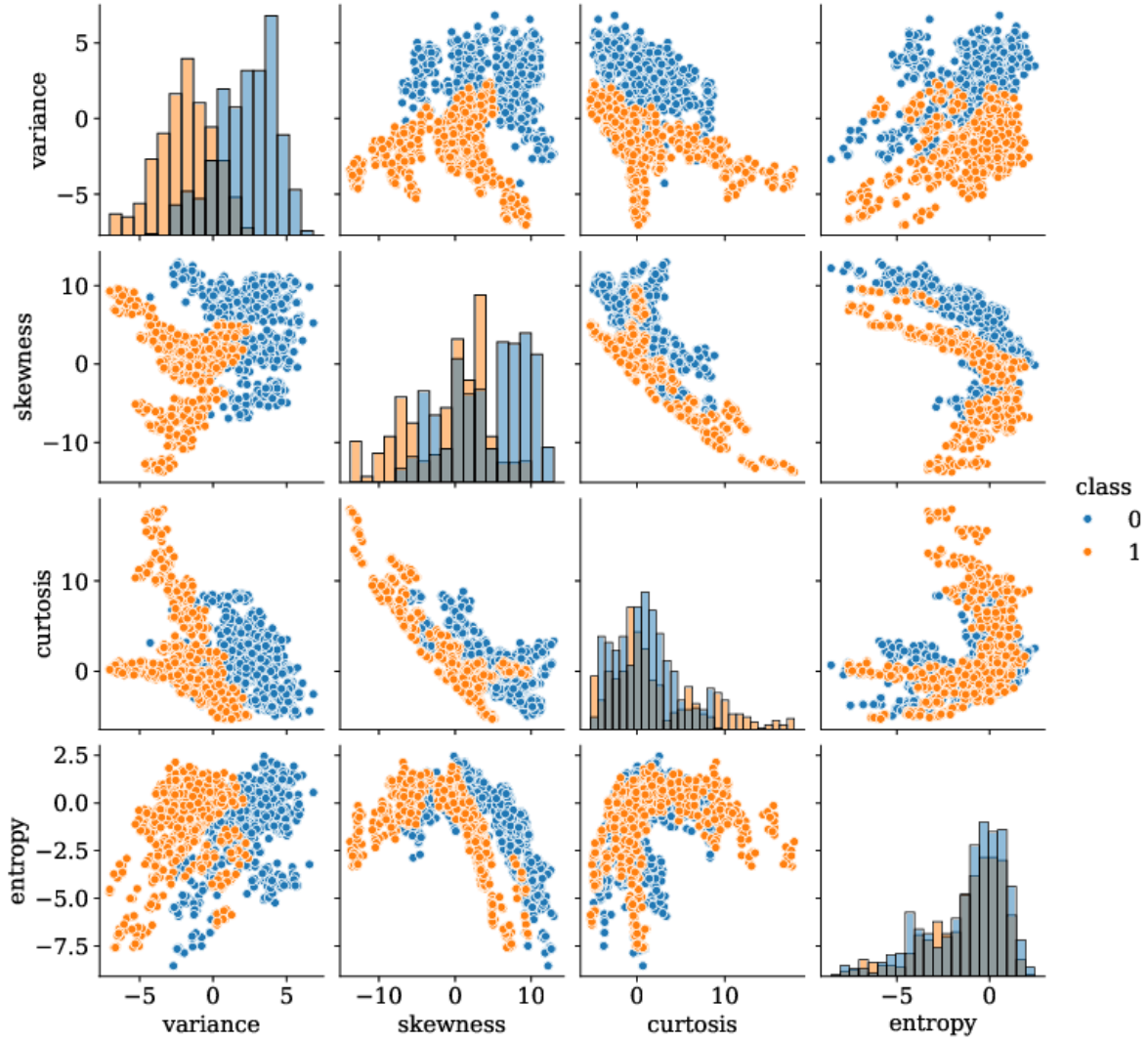


Figure 3: Scatter plots showing pairwise feature relationships coloured by class.

Inference: From the scatter plots, the correlation between the feature pairs for both genuine and fake bank notes can be seen. Almost all the feature pairs have some form of separation showing that the dataset is linearly separable hence suitable for use in a Single Layer Perceptron.

8.4 Training Error vs Epoch

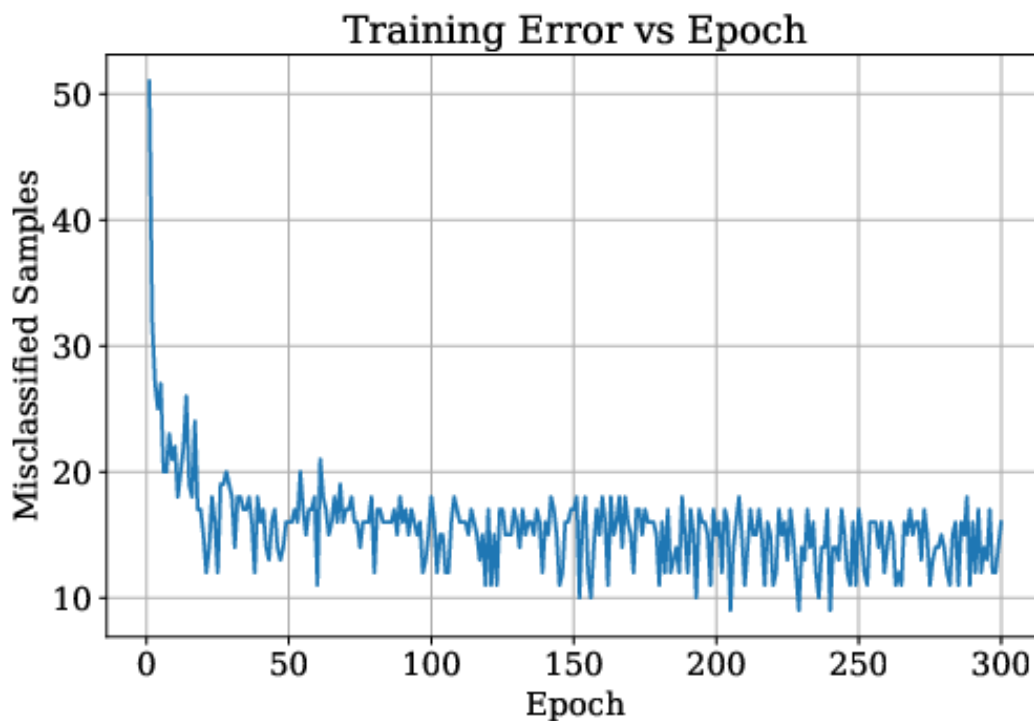


Figure 4: Training error as a function of training epochs.

Inference: Misclassified samples start decreasing very fast in the initial few epochs but gradually tend to stabilize. The pattern shows that the perceptron manages to learn the decision boundary after going through a certain number of iterations.

8.5 Weight Evolution

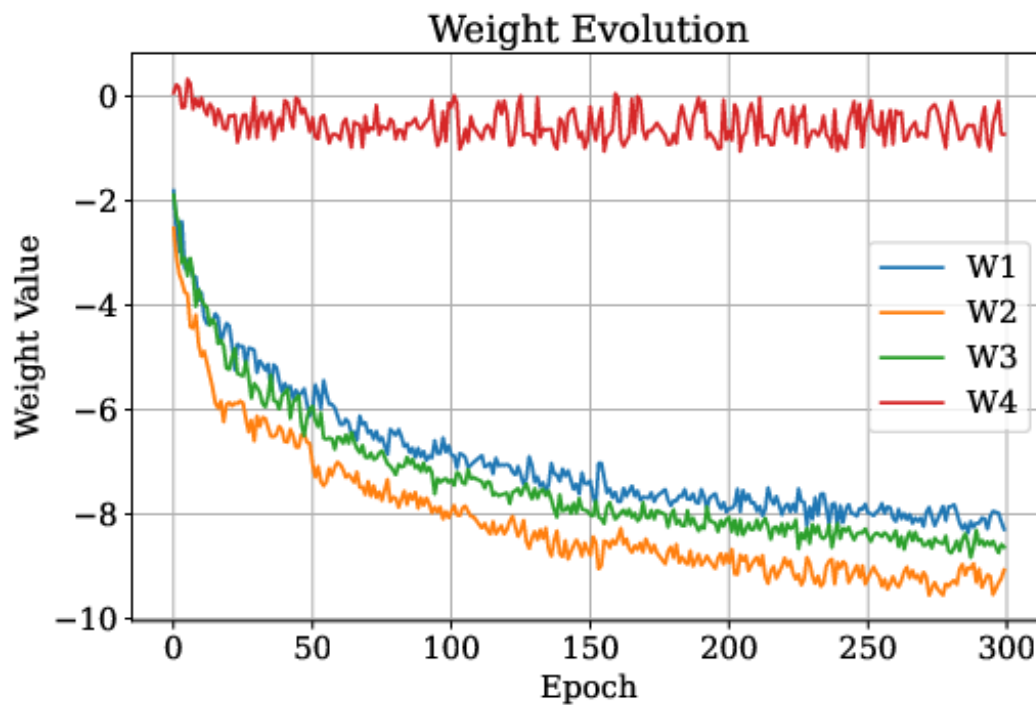


Figure 5: Evolution of the perceptron weights during training.

Inference: Weights undergo substantial change in the initial phases of training and stabilize after some epochs. It shows that the learning process converges to the optimal value of parameters for classification of the two classes.

8.6 Bias Evolution

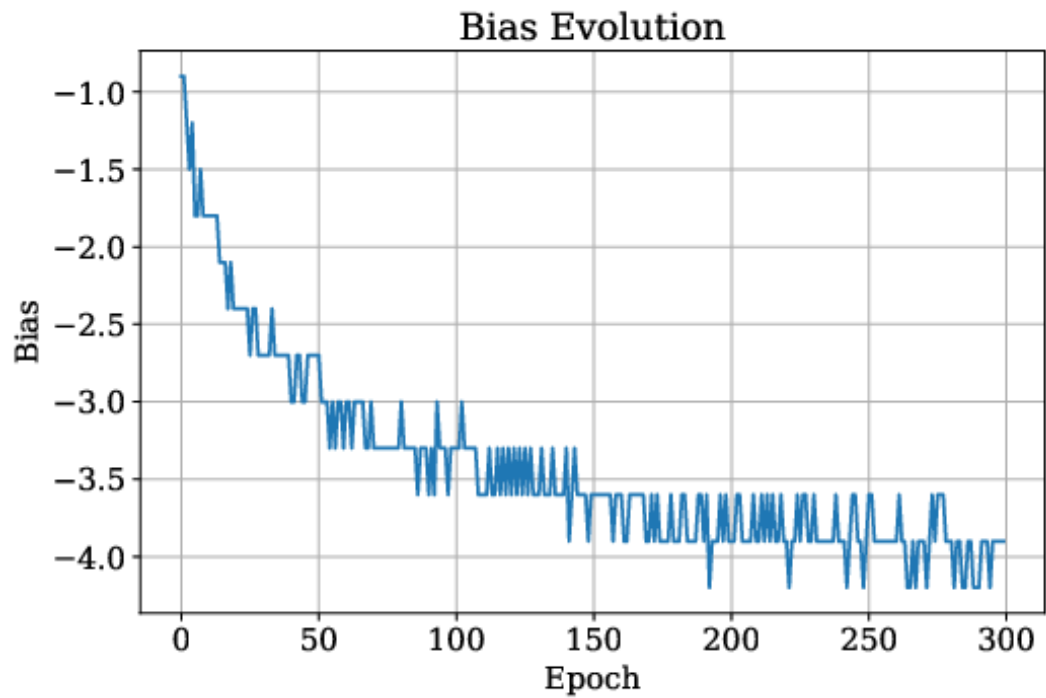


Figure 6: Evolution of the bias parameter during training.

Inference: Bias is continuously updated at each training epoch in conjunction with weights until it becomes almost constant. Bias acquired by the perceptron through training assists in adjusting the decision boundary.

8.7 Confusion Matrix

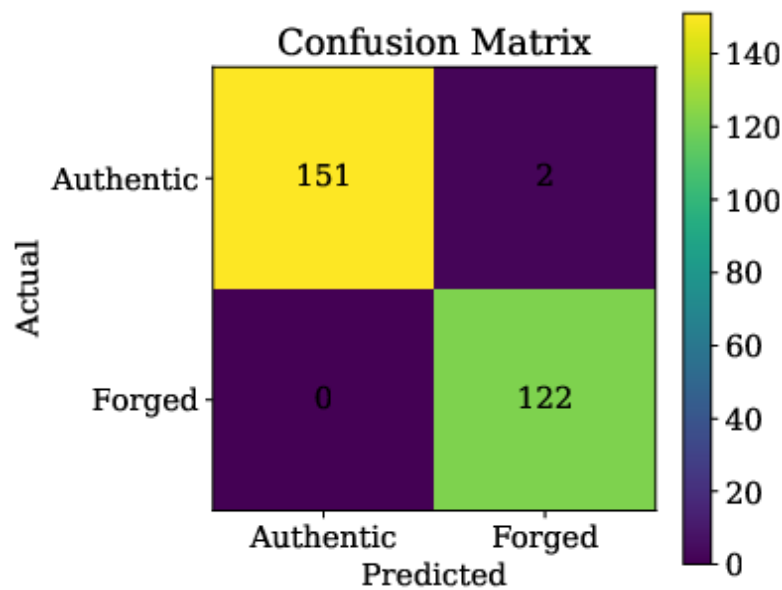


Figure 7: Confusion matrix obtained on the test dataset.

Inference: The confusion matrix gives an overview of the performance of the classifier by listing the numbers of True Positive, True Negative, False Positive and False Negative. The majority of the samples have been classified correctly, thus, implying that the developed perceptron has performed very well.

8.8 Learning Rate Comparison

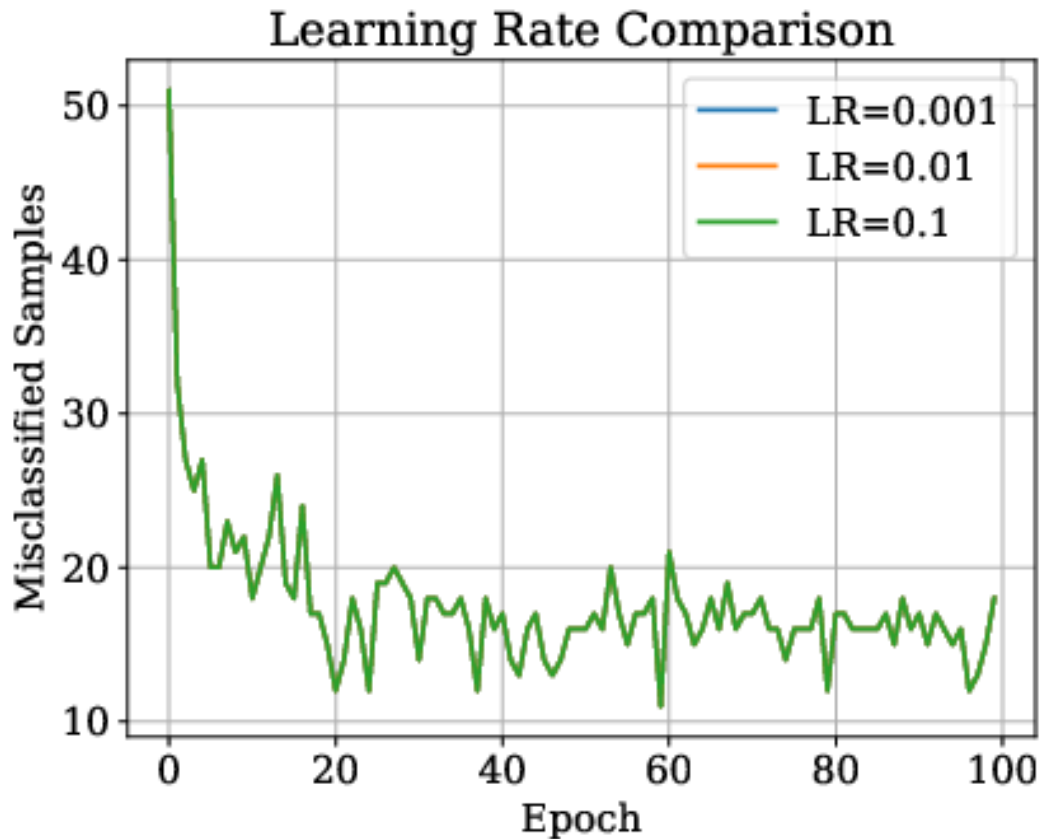


Figure 8: Comparison of convergence for different learning rates.

Inference: The learning rate affects the speed of convergence of the perceptron. When the learning rate is very small, then learning will be slow. If the learning rate is too high, it can lead to unstable updates. In this particular experiment, the learning rate 0.01 worked well.

8.9 Decision Boundary (Optional)

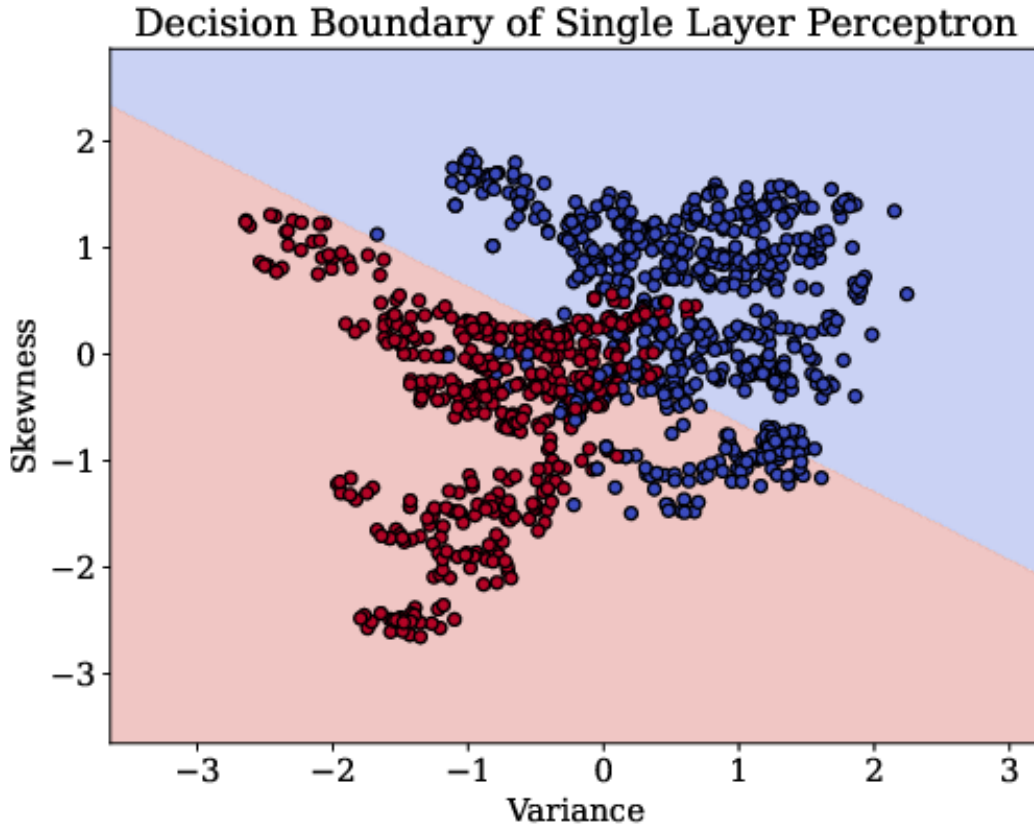


Figure 9: Decision boundary learned by the Single Layer Perceptron using the selected two features.

Inference: The decision boundary is the linear separating hyperplane formed by the Single Layer Perceptron with the use of two chosen features. It can be seen that most of the genuine and counterfeit notes have been classified into two separate areas, suggesting that the chosen features are almost linearly separable. Some samples fall near the decision boundary, which might cause some misclassification errors, though.

9. Performance Tables

9.1 Training Summary

Parameter	Value
Dataset Size	1372 Samples
Train/Test Split	80% Training (1097 samples), 20% Testing (275 samples)
Learning Rate	0.3
Epochs	300
Final Weights	$[-8.2890, -9.0699, -8.6309, -0.7295]$
Final Bias	-3.9
Accuracy	99.27%
Precision	98.39%
Recall	100.00%
F1-score	99.19%

Table 1: Training Summary of the Single Layer Perceptron

9.2 Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	51	-1.8075	-2.5223	-0.9
50	17	-6.4979	-7.3929	-3.6
100	18	-7.3025	-7.9359	-3.9
150	17	-7.6158	-8.2082	-3.9
200	16	-7.8789	-8.5537	-3.9
250	16	-8.1324	-8.8120	-3.9
300	16	-8.2890	-9.0699	-3.9

Table 2: Epoch-wise learning progress of the perceptron

10. Discussion

10.1 Comparison of Step and Sigmoid Activation Functions

The output of the Step Activation Function is always binary (either 0 or 1). On the other hand, the output of the Sigmoid Activation Function lies in the range $[0, 1]$. The Step function is appropriate for the classical perceptron as it can perform binary classification. However, the Sigmoid function is commonly used in modern neural networks due to the probabilistic nature of the output and efficient gradient optimization techniques.

The Step Activation Function is not suitable for use in modern deep learning because of two reasons. First, the Step Activation Function is not differentiable at the threshold point. Second, the gradient of

the function is zero everywhere else. Therefore, there is no way to propagate gradients throughout the network during backpropagation.

Property	Step Function	Sigmoid Function
Output Range	$\{0, 1\}$	$(0, 1)$
Differentiable	No	Yes
Gradient Available	No	Yes
Used In	Classical Perceptron	Modern Neural Networks
Output Type	Binary Class	Probability

10.2 Comparison with Scikit-learn's Perceptron

The perceptron model created in this experiment has been designed from scratch using perceptron learning rules, weight update manually, and step activation function. It assists in comprehending the learning process as well as mathematical calculation behind the perceptron algorithm.

The Scikit-learn perceptron module contains an optimized version of perceptron which is capable of handling data shuffling, early stopping, regularization, and efficient optimization methods automatically. Though both the algorithms generate a linear classifier but the perceptron version in Scikit-learn is more efficient from computational point of view.

10.3 Effect of Different Learning Rates

The perceptron model was trained using learning rates of 0.001, 0.01 and 0.1. The observed test accuracies obtained from the experiment are shown below.

Learning Rate	Test Accuracy
0.001	98.55%
0.01	98.55%
0.1	98.55%

Table 3: Performance for different learning rates

While all three rates of learning were able to achieve the same level of accuracy when tested on the testing data, the manner in which each converged varied. A learning rate of 0.001 led to slow updates of the parameters, while that of 0.1 converged faster but had higher updates for the weights.

10.4 Why a Single Layer Perceptron Cannot Solve the XOR Problem

XOR is a non-linearly separable problem because a straight line cannot be used to discriminate between the two classes of output. The Single Layer Perceptron cannot be used to learn multiple decision boundaries because it is capable of learning only one decision boundary.

In order to address the XOR problem, there needs to be at least one layer of hidden neurons with a non-linear activation function. This is where Multilayer Perceptrons (MLP) excel.

10.5 Effect of Feature Normalization

Prior to the learning process, all the numerical attributes in the data were scaled through the use of the **StandardScaler**. Feature normalization made sure that all the features had the same scaling so that those features with larger numerical values did not overshadow other features during learning.

The normalization led to smooth convergence of the perceptron and also led to accurate weight updates and high prediction accuracy. The performance of the classifier was around 99.27%.

11. Conclusion

In this experiment, a Single Layer Perceptron was developed from scratch by coding it in Python, without using any machine learning libraries to implement the learning algorithm. The perceptron was trained using the Banknote Authentication dataset, where the dataset was first preprocessed, split between train and test datasets, and normalized before training.

The Exploratory Data Analysis (EDA) was done using histograms, scatter plots, box plots, and a correlation heatmap for exploring the characteristics of the dataset. The training procedure was explored via plotting of training error, weight and bias evolution, comparison of learning rates, and plotting of decision boundaries.

The trained model was found to be highly accurate at around **99.27%** on the test set with good precision, recall, and F1-score. Furthermore, the confusion matrix confirmed that there were no many mistakes in the classification process.

Generally, from this experiment, a proper understanding of the working principles of Single Layer Perceptron was gained. These include linear decision boundary, weights update, learning rate, and feature normalization. In addition, it was learned that a perceptron can be effective in solving the linearly separable classification tasks but it cannot solve non-linearly separable sets like the XOR problem. All the above gained knowledge is the stepping stone to studying more advanced network architectures like MLPs and Deep Neural Networks.

12 Additional Task

12.1 Objective

Task: Code the Perceptron Learning Algorithm for OR, NOT, and AND gates using Python. Show the weights after each update. Plot the decision boundary after each weight update using Python built-in packages.

12.2 Generated Decision Boundary Plots

12.2.1 AND Gate

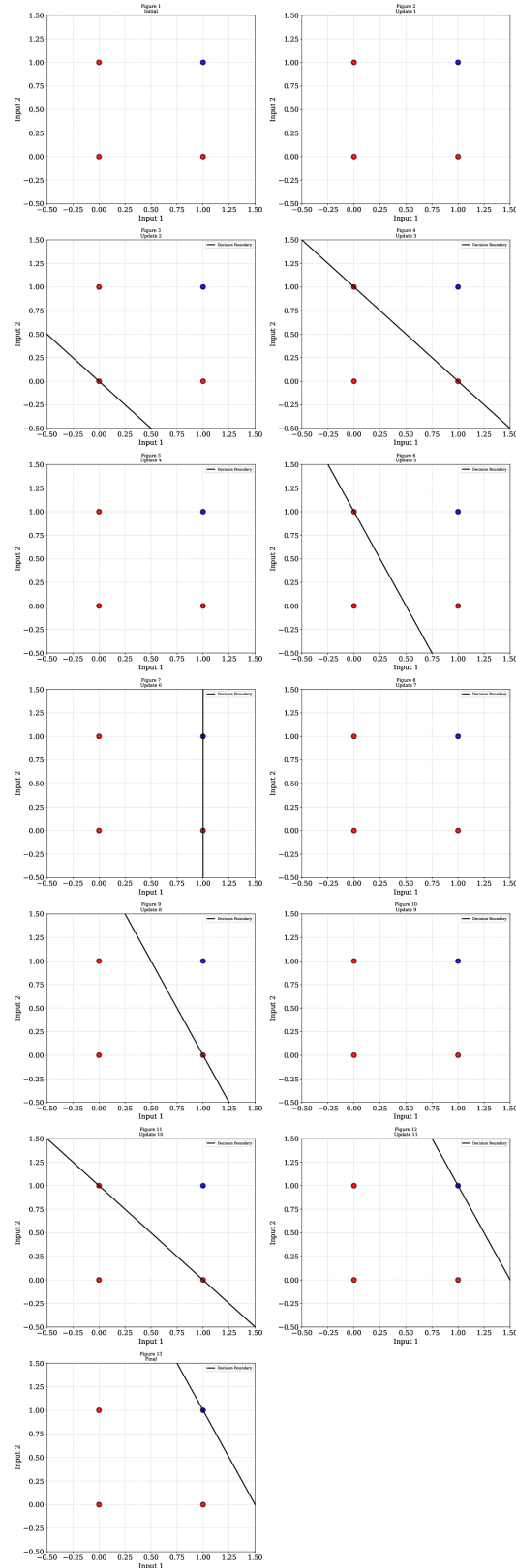


Figure 10: Decision boundary evolution for the AND gate after every weight update.

12.2.2 OR Gate

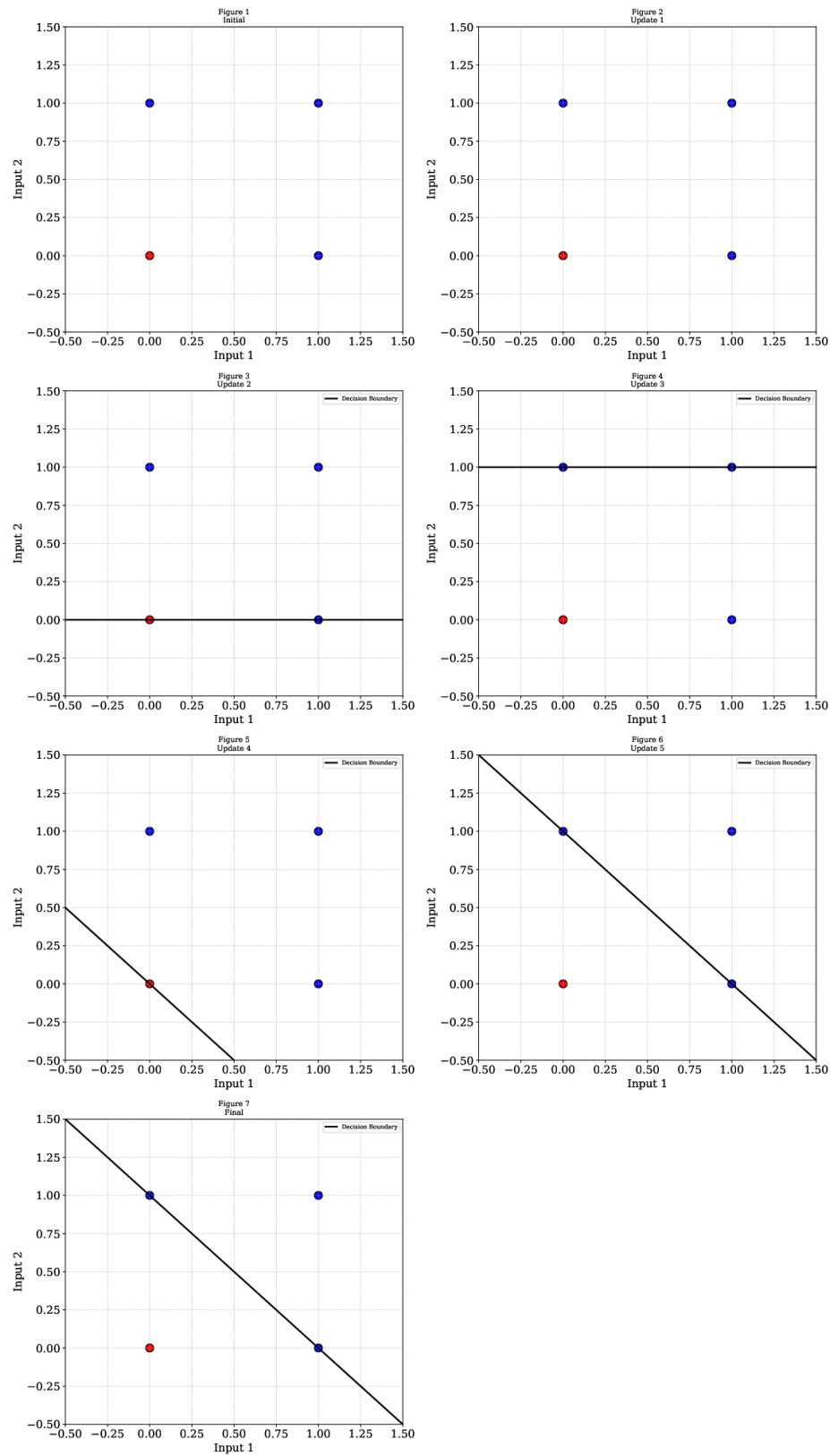


Figure 11: Decision boundary evolution for the OR gate after every weight update.

12.2.3 NOT Gate

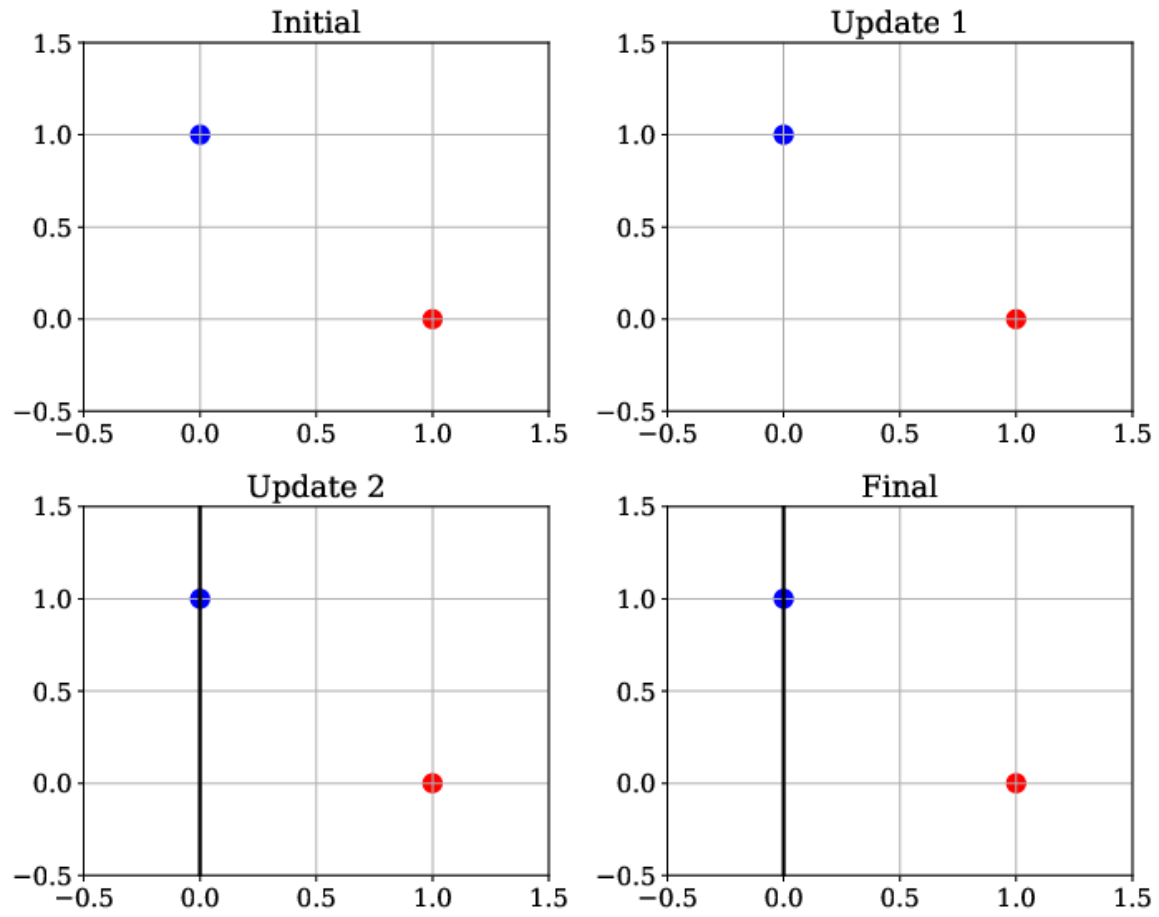


Figure 12: Decision threshold evolution for the NOT gate after every weight update.

12.3 Weight Update Tables

12.3.1 AND Gate

Table 4: Weight updates for the AND gate

Update	w_1	w_2	Bias
1	0	0	-1
2	1	1	0
3	1	1	-1
4	1	0	-2
5	2	1	-1
6	2	0	-2
7	1	0	-3
8	2	1	-2
9	1	1	-3
10	2	2	-2
11	2	1	-3

Final Weights: (2, 1)

Final Bias: -3

12.3.2 OR Gate

Table 5: Weight updates for the OR gate

Update	w_1	w_2	Bias
1	0	0	-1
2	0	1	0
3	0	1	-1
4	1	1	0
5	1	1	-1

Final Weights: (1, 1)

Final Bias: -1

12.3.3 NOT Gate

Table 6: Weight updates for the NOT gate

Update	Weight	Bias
1	-1	-1
2	-1	0

Final Weight: -1

Final Bias: 0

12.4 Conclusion

The Perceptron Learning Algorithm has been effectively executed for the AND, OR and NOT logic gates. The weights and bias were adjusted for every misclassification of a training sample in order to demonstrate the learning process of the perceptron. The decision boundary plots that have been generated for the AND and OR gates reveal how the separating hyperplane slowly approaches the point where all training samples will be correctly classified. The reason for which the NOT gate has only one input variable is that the learning process is depicted by the decision threshold rather than a decision boundary.

13. Report Contents

The report should include:

- Objective
- Background Theory
- Activation Functions
- Dataset Description
- Experimental Procedure
- Source Code
- Experimental Results
- Generated Plots with Interpretation
- Performance Tables
- Discussion
- Conclusion
- References

14. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.